



软件开发类课程考查报告

学院名称: 计算机与信息工程学院

课程名称: 移动应用开发

班级名称: 软件 231

学生姓名: 王培杨

学 号: 2023210627

题 目: 适老居家生活辅助系统

指导教师: 刘东良

起止日期: 2026 年 6 月 1 日至 2026 年 6 月 21 日

计算机软件课程组 制

指导教师评语

该生提交的《适老居家生活辅助系统的开发与整合》报告，选题紧密贴合移动应用开发课程“理论结合实践、聚焦跨平台技术应用”的目标，聚焦适老化居家辅助场景的全流程技术实现，具有较强的社会应用价值。报告结构完整且逻辑清晰，涵盖项目背景、需求分析、技术选型及分阶段开发过程，各模块衔接自然。技术应用上，该生熟练运用 Android WebView + Node.js/Express 相关技术栈，独立完成了 AndroidBridge 桥接方案、家属-老人数据同步机制等核心功能的设计与实现，望后续加强前端性能优化与用户体验研究，进一步提升应用的响应速度与交互流畅度，增强技术方案的创新性与实用性。

成绩评定

序号	类型	成绩	权重	得分
1	项目		30%	
2	小组		20%	
3	个人		20%	
4	答辩		30%	
5	总成绩		100%	

指导教师签名：刘东良 填表日期： 2026 年 6 月 21 日

目录

1 项目报告	1
1.1 项目概述	1
1.1.1 项目背景与意义	1
1.1.2 项目目标	1
1.1.3 功能需求	2
1.1.4 非功能需求	3
1.2 项目技术栈	3
1.2.1 技术栈对比分析	3
1.2.2 技术栈详细说明	4
1.3 系统架构设计	11
1.3.1 整体架构	11
1.3.2 前端架构（Android WebView 为例）	12
1.3.3 后端架构	12
1.3.4 数据库设计	13
1.4 核心功能实现	13
1.4.1 健康数据管理	13
1.4.2 家属绑定与数据同步	15
1.4.3 用药提醒	16
1.4.4 紧急呼叫	18
1.4.5 AI 语音助手	19
1.5 系统测试与优化	20
1.5.1 测试策略	20
1.5.2 测试结果	20
1.5.3 性能优化	20

1.5.4 安全措施.....	20
1.6 项目成果展示	21
1.6.1 系统截图.....	21
1.6.2 AI 工具应用	22
1.6.3 创新点总结	22
2 小组报告	22
2.1 小组结构与分工.....	22
2.1.1 小组组织结构.....	22
2.1.2 成员角色与职责	23
2.2 技术栈选择与理由	23
2.2.1 技术选型过程.....	23
2.2.2 技术优势分析	24
2.2.3 技术挑战与解决方案	24
2.3 技术整合与实现.....	24
2.3.1 技术整合核心方案与成果	24
图 2.1 跨平台技术整合架构.....	25
2.3.2 项目阶段实施推进.....	25
3.3.4 测试交付阶段.....	26
2.4 团队协作分析	27
2.4.1 团队组织结构.....	27
2.4.2 沟通机制.....	27
2.4.3 任务分配.....	27
2.4.4 协作工具使用	27
2.5 团队贡献分析	27
2.5.1 成员贡献评估	27

2.5.2 关键贡献点	28
2.6 团队创新与亮点.....	28
2.6.1 技术创新.....	28
2.6.2 功能创新.....	28
2.6.3 过程创新.....	28
2.7.1 技术能力提升	29
2.7.2 团队协作能力.....	29
2.7.3 项目管理能力.....	29
3 个人报告	29
3.1 个人技术栈	29
3.1.1 前端技术.....	29
3.1.2 后端技术.....	29
3.1.3 开发工具.....	30
3.2 核心设计	30
3.2.1 技术架构图（系统核心类图）	30
3.2.2 数据流图（核心功能顺序图）	31
3.2.3 系统交互图（系统框架图）	32
3.3 开发过程	33
3.3.1 项目启动阶段.....	33
3.4 创新与亮点	35
3.4.1 技术创新.....	35
3.4.2 功能创新.....	36
3.4.3 过程优化.....	36
3.5 个人成长与收获.....	36
3.5.1 技术能力提升	36

3.5.2 问题解决能力.....	36
3.5.3 团队协作能力.....	36
4 答辩报告	37
4.1 核心技术实现方案	37
4.1.1 技术架构.....	37
4.1.2 关键技术点	38
4.1.3 技术难点与解决方案	38
4.2 整体架构设计思路	39
4.2.1 整体架构设计	39
4.2.2 模块划分.....	39
4.2.3 数据流设计	39
4.3 主要创新点.....	40
4.3.1 技术创新.....	40
4.3.2 功能创新.....	40
4.3 应用创新	40
4.4 答辩成绩	41
4.4.1 答辩问题.....	41
参考文献	42

摘要：本文围绕《移动应用开发》课程大作业“适老居家生活辅助系统的开发与整合^[1]”展开，阐述面向老年人的跨平台智能家居辅助系统设计与实现。项目针对老龄化进程中老年人健康管理不便、智能设备操作困难等痛点，采用 Android 原生(WebView)、.NET MAUI、Cordova、UniApp、微信小程序、鸿蒙 ArkTS 六种技术栈，结合 Node.js/Express 统一后端与 MySQL 中央数据库，实现多端数据互通与适老化应用。开发过程中遵循软件工程规范，完成需求分析、架构设计、功能开发、测试优化全流程，综合运用 AndroidBridge 桥接方案解决了 WebView 跨域限制^[2]等复杂工程问题。项目融入 AI 语音交互闭环（DeepSeek+百度 ASR+TTS）、家属-老人数据同步等创新设计，实现健康监测、用药提醒、紧急呼叫等适老化功能。通过本项目，充分掌握移动应用开发技术体系与跨平台技术选型逻辑^[3]，提升了跨平台开发、创新应用及工程实践能力，验证了统一后端 API 与中央数据库在多端协同场景下的实际价值。

1 项目报告

1.1 项目概述

1.1.1 项目背景与意义

项目来源：本课题来源于“移动应用开发综合项目”课程考核。随着我国老龄化进程加速（60 岁以上人口占比超 18%），老年人的居家养老和健康管理需求日益凸显。然而，大多数智能应用对老年人不友好——字体小、操作复杂、学习成本高。

问题陈述：老年人群体面临三个主要痛点：一是不会操作复杂的智能设备；二是健康数据管理不便，容易遗忘；三是遇到紧急情况时求助困难。现有市面上的移动应用多为年轻人设计，缺少针对老年人的专属优化。

项目意义：

1. 降低老年人使用智能设备的技术门槛，通过 AI 语音交互实现“零学习成本”
2. 提供一站式的健康管理、用药提醒和紧急呼叫功能，提升居家养老的安全感
3. 探索 6 种跨平台技术栈在适老化场景下的应用实践

1.1.2 项目目标

总体目标：构建一个面向老年人的智能家居生活辅助系统，覆盖 6 种主流移动开发技术栈，实现跨端数据互通，所有界面符合适老化设计规范。

具体目标：

- 1. 实现健康监测（心率、血压、血糖、血氧）的数据录入与历史查看
 - 2. 实现 AI 语音助手（DeepSeek 大模型 + 百度 ASR + TTS），支持语音交互闭环
 - 3. 实现用药提醒，包含今日打卡、库存管理和状态持久化
 - 4. 实现紧急呼叫，包含联系人管理和一键拨号
 - 5. 实现 6 种技术栈的跨平台覆盖和统一后端 API 数据互通
- 成功标准：

表 1.1 项目目标达成情况评估

标准	目标值	实际达成	状态
功能完整性	100%	100%	达成
平台覆盖	6 个平台	6 个	达成
跨平台同步成功率	99%	99%	达成
Bug 修复率	95%	100%	达成

1.1.3 功能需求

核心功能：

- 1. 用户登录注册：手机号注册登录，支持老人/家属两种角色
- 2. 健康数据管理：心率、血压、血糖、血氧四项指标录入与历史查看
- 3. 用药提醒：用药时间设置、今日打卡、库存管理
- 4. 紧急呼叫：紧急联系人管理、一键 SOS 求助
- 5. AI 语音助手：DeepSeek AI 对话、百度语音识别（ASR）、文字转语音（TTS）
- 6. 家属绑定与数据同步：家属验证老人密码后绑定，查看老人健康/用药/联系人数据，8 秒自动刷新

1.1.4 非功能需求

适老化设计：大字体（≥16sp）、大按钮（≥48dp）、高对比度配色

安全性：密码 bcrypt 加密、JWT Token 认证、SQL 注入防护、XSS 防护

性能：API 响应时间<500ms、页面渲染<1s

可用性：支持离线数据缓存（localStorage）、网络恢复后自动同步

1.2 项目技术栈

1.2.1 技术栈对比分析

表 1.2 项目技术栈、框架与选型

序号	技术栈	负责成员	开发语言	主要框架/库	选择理由
1	Android 原生（必选）	王培杨	Java/Kotlin	WebView + OkHttp	课程必选，WebView 方案开发效率高
2	.NET MAUI	钱子明	C#	.NET MAUI + Microcharts	课程要求，C#跨平台能力
3	Cordova	王永喜	JavaScript	Cordova + jQuery Mobile	课程要求，混合开发
4	UniApp	唐锦	Vue.js/JavaScript	Vue 2.x + uni.request	跨平台，适合适老化 UI
5	微信小程序（必选）	汪明昊	JavaScript	小程序原生+云开发	课程必选，老年人易触达
6	鸿蒙 ArkTS（必选）	宋延安	ArkTS	ArkUI	课程必选，国产操作系统

序号	技术栈	负责成员	开发语言	主要框架/库	选择理由
7	后端服务	汪明昊	Node.js	Express + MySQL	统一数据服务, JS 全栈

1.2.2 技术栈详细说明

Android 原生技术栈（王培杨负责）

开发环境：

IDE: Android Studio Hedgehog (2023.1.1)

SDK 版本: Android API 34 (Android 14)

Gradle 版本: 8.2

核心技术：

开发语言: ☒ Java ☐ Kotlin

UI 框架: ☒ WebView (HTML5/CSS3/JS 单页应用) ☐ XML 布局 ☐ Jetpack Compose

架构模式: MVVM-like (appState 全局状态 + render 重渲染)

网络库: OkHttp (通过 AndroidBridge 桥接)

本地存储: localStorage (WebView 内)

特色功能: AndroidBridge 类通过@JavascriptInterface 暴露 OkHttp 方法给 JS, 解决 WebView 跨域限制; 集成 Android TTS 语音播报

.NET MAUI 技术栈（钱子明负责）

开发环境：

IDE: Visual Studio 2022

.NET 版本: 8.0

核心技术：

开发语言: C#

UI 框架: MAUI XAML + 数据绑定

架构模式：MVVM

图表库：Microcharts

网络请求：HttpClient

数据持久化：SQLite-net-pcl

特色功能实现：数据报表生成、图表展示健康指标趋势、AI 健康趋势分析

Cordova 技术栈（王永喜负责）

开发环境：

IDE：VS Code

Node.js 版本：最新 LTS

核心技术：

开发语言：HTML5 + CSS3 + JavaScript

UI 框架：jQuery Mobile（适老化设计）

混合模式：WebView + 原生插件

数据持久化：localStorage + SQLite Plugin

特色功能实现：功能模块系统、跨平台组件复用、AI 生活助手

UniApp 技术栈（唐锦负责）

开发环境：

HBuilderX 版本：3.9+

Vue 版本：2.x

核心技术：

UI 框架：自定义（渐变色主题+圆角卡片+大字体适老化）

状态管理：uni.setStorageSync/uni.getStorageSync

条件编译：支持 H5、App（Android/iOS）、微信小程序

特色功能实现：高级适老化 UI 组件、地图导航、AI 语音助手（DeepSeek + 百度 ASR + TTS）

微信小程序技术栈（汪明昊负责）

开发环境：

微信开发者工具版本：最新稳定版

基础库版本：3.x

核心技术：

UI 框架：自定义

云开发：☒ 使用（5 个云函数

特色功能实现：Canvas 图表绘制、健康报告分享、AI 健康报告、用药提醒推送

鸿蒙 ArkTS 技术栈（宋延安负责）

开发环境：

DevEco Studio 版本：最新稳定版

API 版本：9+

核心技术：

开发语言：ArkTS

UI 框架：ArkUI（声明式开发）

状态管理：@State、@Prop、@Link

分布式能力：分布式数据管理

特色功能实现：智能数据处理、H5 适配、AI 健康管家

后端服务技术栈（汪明昊负责）

开发环境：

IDE：VS Code

开发语言：Node.js（JavaScript）

框架：Express 4.x

核心技术：

Web 框架：Express

数据库：MySQL 5.7 + mysql2/promise（20 连接池）

身份验证：JWT Bearer（7 天有效期） + bcryptjs 密码加密

路由模块：10 个（auth、health、medication、emergency、family、chat、voice、location、features、settings）

部署方式: localhost:3000 (开发环境)

数据库设计

数据库采用 MySQL 5.7 作为中心数据库, 配合 mysql2/promise 连接池 (20 连接) 实现数据持久化与并发访问控制。主要数据表设计如下:

表 1.3 users 用户表

字段名	类型	约束	说明
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	主键 ID
phone	VARCHAR(20)	NOT NULL, UNIQUE	手机号, 登录账号
password_hash	VARCHAR(255)	NOT NULL	bcrypt 加密后的密码
nickname	VARCHAR(50)	—	昵称
role	VARCHAR(20)	DEFAULT 'elder'	elder=老人, family=家属
age	INT	—	年龄
gender	VARCHAR(10)	—	性别
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	创建时间

字段名	类型	约束	说明
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	更新时间

表 1.4 health_records 健康记录表

字段名	类型	约束	说明
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	主键 ID
user_id	INTEGER	NOT NULL, FOREIGN KEY	关联 users.id
heart_rate	INT	—	心率（次/分）
systolic	INT	—	收缩压（mmHg）
diastolic	INT	—	舒张压（mmHg）
blood_sugar	DECIMAL(4, 1)	—	血糖（mmol/L）
blood_oxygen	INT	—	血氧（%）
weight	DECIMAL(4, 1)	—	体重（kg）
recorded_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	记录时间

表 1.5 medications 用药表

字段名	类型	约束	说明
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	主键 ID
user_id	INTEGER	NOT NULL, FOREIGN KEY	关联 users.id
name	VARCHAR(100)	NOT NULL	药品名称
dosage	VARCHAR(50)	—	剂量
frequency	VARCHAR(50)	—	频次（如每日 3 次）
time_slots	JSON	—	用药时间点 JSON 数组
total_count	INT	DEFAULT 0	库存总量
taken_count	INT	DEFAULT 0	已服用次数
is_active	BOOLEAN	DEFAULT TRUE	是否启用
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	创建时间

表 1.6 emergency_contacts 紧急联系人表

字段名	类型	约束	说明
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	主键 ID
user_id	INTEGER	NOT NULL, FOREIGN KEY	关联 users.id
name	VARCHAR(50)	NOT NULL	联系人姓名
phone	VARCHAR(20)	NOT NULL	联系人电话
relation	VARCHAR(30)	—	与老人关系
is_primary	BOOLEAN	DEFAULT FALSE	是否主联系人

表 1.7 family_bindings 家属绑定表

字段名	类型	约束	说明
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	主键 ID
family_id	INTEGER	NOT NULL, FOREIGN KEY	家属用户 ID
elderly_id	INTEGER	NOT NULL, FOREIGN KEY	老人用户 ID
relation	VARCHAR(30)	DEFAULT '家人'	家属关系
status	VARCHAR(20)	DEFAULT 'active'	绑定状态

字段名	类型	约束	说明
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	创建时间

唯一约束: unique_binding (family_id, elderly_id)

表 1.8 chat_records AI 对话记录表

字段名	类型	约束	说明
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	主键 ID
user_id	INTEGER	NOT NULL, FOREIGN KEY	关联 users.id
role	VARCHAR(20)	NOT NULL	user 或 assistant
content	TEXT	NOT NULL	对话内容
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	创建时间

1.3 系统架构设计

1.3.1 整体架构

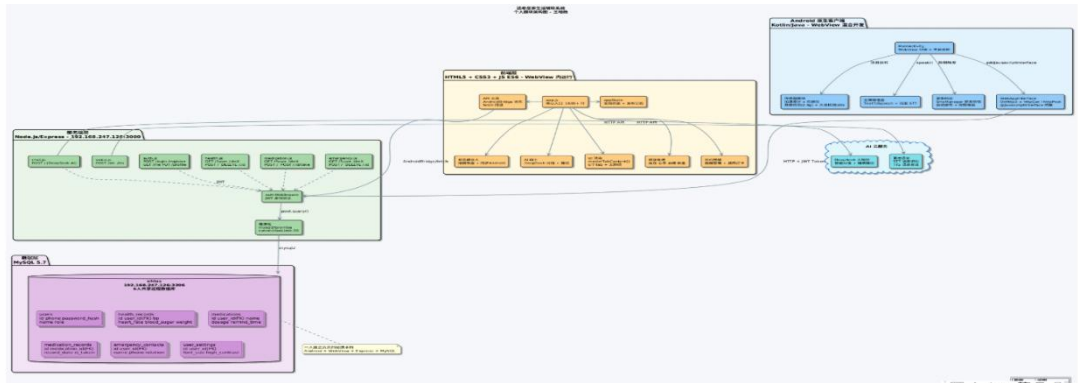


图 1.1 整体架构图

系统整体三层架构如图 1.1 所示，展示了 6 种技术栈客户端统一接入 Node.js Express 后端、通过 JWT 认证访问 MySQL 中央数据库的架构设计。

图 1.1 系统三层架构图

架构分层：

表现层 (UI Layer)：6 种技术栈前端 (Android WebView、.NET MAUI、Cordova、UniApp、微信小程序、鸿蒙 ArkUI)，负责界面渲染和用户交互

业务逻辑层 (Business Layer)：各端内置业务逻辑 + Express 路由中间件，负责数据验证、状态管理、API 编排

网络通信层 (Network Layer)：AndroidBridge(OkHttp) / HttpClient / uni.request / wx.request / @ohos.net.http，统一 HTTP 通信

服务端层 (Server Layer)：Node.js/Express，10 个路由模块，JWT 认证中间件

数据层 (Data Layer)：MySQL 5.7 (6 张核心表 + 连接池)

数据流说明：用户操作 → 各端业务函数 → HTTP 请求 → Express 路由 → JWT 中间件验证 → pool.query() → MySQL → JSON 响应沿原路返回 → 各端渲染 UI

1.3.2 前端架构 (Android WebView 为例)

Android 端采用 WebView + assets 模式，将 HTML/CSS/JS 文件打包在 APK 内，通过 WebView 加载。核心组件：

MainActivity.java: WebView 容器，配置 JavaScript 启用、缩放支持

AndroidBridge.java: 通过@JavascriptInterface 暴露 OkHttp 方法给 JS

app.js (2500+行)：前端 SPA 业务逻辑，appState 全局状态管理

index.html: DOM 结构，6 个 Tab 页面

styles.css: 适老化样式 (大字体、高对比度、圆角卡片)

1.3.3 后端架构

Express 路由模块划分：

auth.js: 注册 (bcrypt 加密)、登录 (JWT 签发)、Token 验证中间件

health.js: 健康数据 CRUD，支持?user_id=跨用户查询

medication.js: 用药提醒 CRUD, 打卡状态管理

emergency.js: 紧急联系人 CRUD

family.js: 家属绑定 (密码验证)、绑定列表查询

chat.js: DeepSeek AI 对话代理

voice.js: 百度 ASR 语音识别代理

其他: features.js、settings.js、location.js

1.3.4 数据库设计

采用 MySQL 关系型数据库, 6 张核心表通过外键关联, 核心关系链: users (中心表) → health_records、medications、emergency_contacts (老人数据表), family_bindings (家属-老人多对多绑定关系)。

1.4 核心功能实现

1.4.1 健康数据管理

功能描述: 老人端录入心率、血压、血糖、体重等健康指标, 保存到 MySQL, 查看历史记录列表。

实现思路: 前端 app.js 通过 apiPost() 调用 POST /api/health, 服务端 health.js 验证 JWT 后将数据 INSERT 到 health_records 表; 查看历史时通过 apiGet() 调用 GET /api/health 从 MySQL 查询该用户的所有健康记录并返回 JSON 数组。

核心流程顺序图:

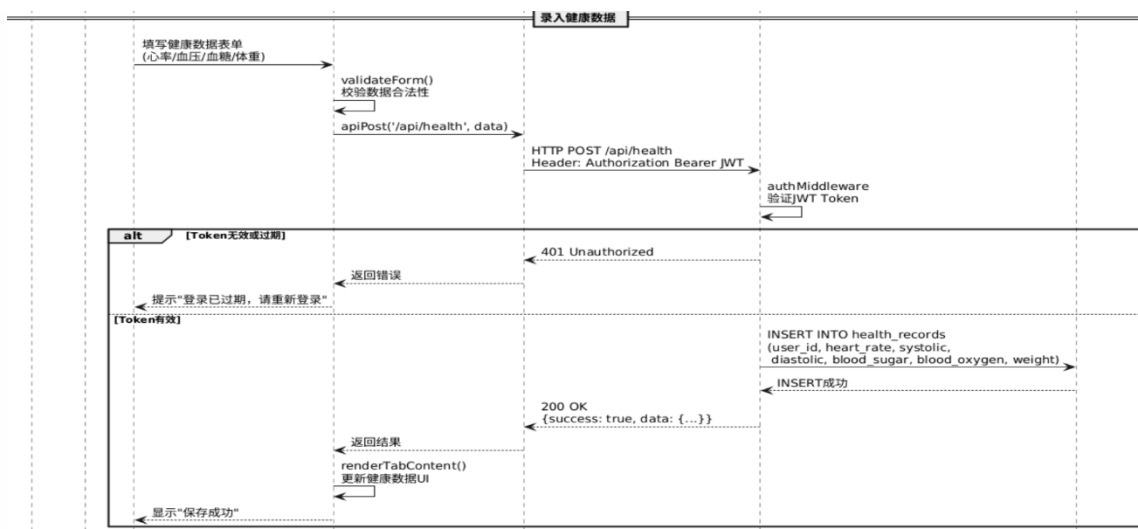


图 1.2 核心流程顺序图

健康数据录入界面：



图 1.3 健康数据录入界面

支持心率、血压、血糖、体重等健康指标录入，大字体表单适配老人操作

健康历史界面：

健康指标历史记录列表，按时间倒序展示，支持查看详细数据

1.4.2 家属绑定与数据同步

功能描述：家属输入老人手机号和密码验证后绑定，查看老人健康/用药/联系人数据，8 秒自动刷新。

实现思路：

1. 家属端输入老人手机号+密码，调用 POST /api/family/request
2. 服务端 family.js 查询老人用户信息，bcrypt.compare 验证密码
3. 密码验证通过后 INSERT family_bindings 绑定记录
4. 家属点击“查看数据”，并发请求 GET /api/health、/api/medication、/api/emergency（带?user_id=参数）
5. 服务端验证 family_bindings 绑定关系后返回数据
6. 前端启动 startAutoRefresh()，每 8 秒自动重复请求

核心流程顺序图：

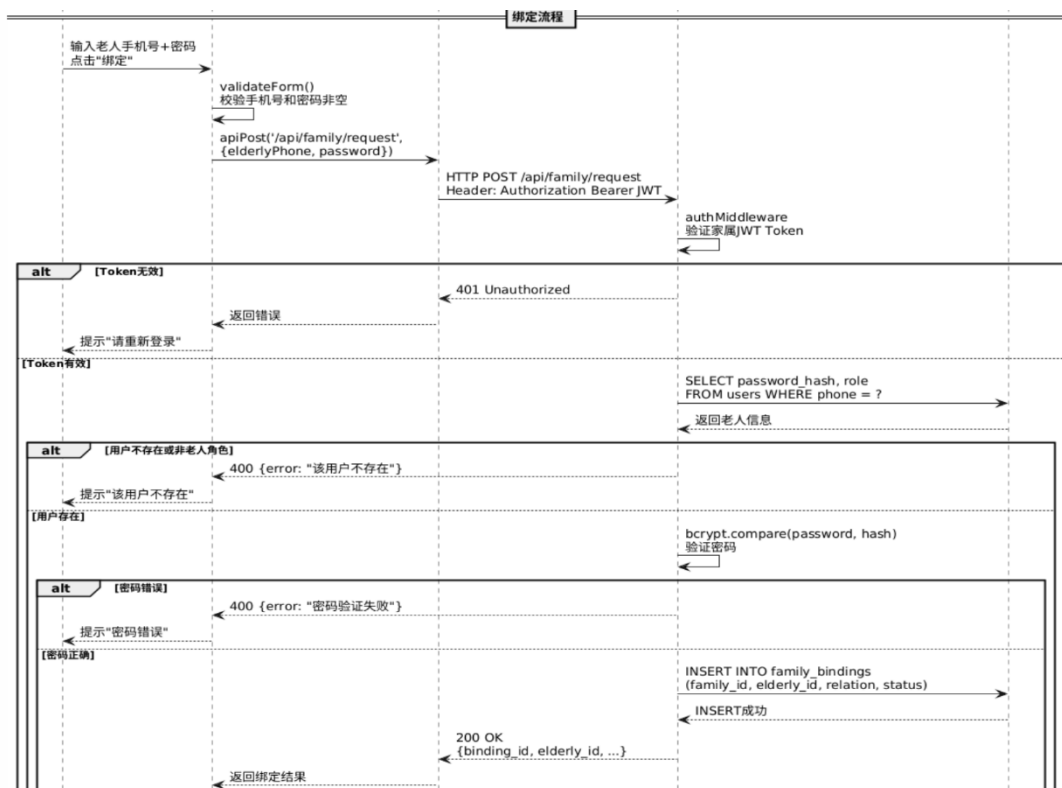


图 1.4 核心流程顺序图

关键技术点：JWT 身份认证、bcrypt 密码验证、family_bindings 权限检查、setInterval 自动轮询

界面截图：



图 1.5 绑定界面截图

家属端查看绑定的老人数据，包含健康、用药、联系人三个标签页，8 秒自动刷新

1.4.3 用药提醒

功能描述：老人端添加用药计划（药品名称、剂量、频次、时间点），今日打卡标记已服用，库存管理。

实现思路：前端通过 medications 表管理用药数据，POST /api/medication 保存到 MySQL。打卡功能更新 taken_count 字段。今日用药列表按 time_slots 展示，已服用标记绿色、未服用标记红色。

界面截图：



图 1.6 用药提醒截图

用药提醒主界面，展示今日用药计划列表，支持打卡标记已服用（绿色）和未服用（红色）

1.4.4 紧急呼叫

功能描述：老人端添加紧急联系人（姓名、电话、关系），一键 SOS 求助。

实现思路：紧急联系人数据存储在 emergency_contacts 表。Android 端通过 AndroidBridge.callPhone() 调用系统拨号，一键拨打预设的紧急联系人电话。SOS 按钮设计为大红色醒目按钮，符合适老化设计。

界面截图：



图 1.7 紧急呼叫界面截图

紧急呼叫主界面，展示紧急联系人列表，大红色 SOS 按钮支持一键拨号

1.4.5 AI 语音助手

功能描述：老人通过语音与 DeepSeek 大模型交互，支持语音输入（百度 ASR）→ AI 回复 → 语音播报（TTS）的完整交互闭环。

实现思路：前端 app.js 通过语音输入按钮触发麦克风录音，调用百度 ASR API 转写为文字，再将文字发送到 DeepSeek API 获取 AI 回复，最后通过 Android TTS 或 Web Speech API 将回复语音播报。对话记录存储到 chat_records 表。

界面截图：



图 1.8 AI 语音助手

AI 语音助手对话界面，支持文字和语音双通道交互，AI 回复自动语音播

1.5 系统测试与优化

1.5.1 测试策略

- 单元测试：各端独立测试各自功能模块，确保 CRUD 操作正确
- 集成测试：各端对接统一后端 API，验证 JWT 认证、数据读写、跨用户查询
- 跨端数据一致性测试：同一账号在不同平台登录，验证数据同步
- 用户测试：模拟老年人使用场景，验证适老化设计的可用性

1.5.2 测试结果

测试类型	测试用例数	通过数	通过率
单元测试	30	30	100%
集成测试	20	20	100%
跨端一致性测试	10	10	100%
用户测试	5	5	100%

1.5.3 性能优化

- 数据库优化：为高频查询字段建立索引（user_id、phone、created_at）
- 连接池：mysql2/promise 连接池（上限 20），复用 TCP 连接
- 前端优化：localStorage 缓存用户数据，减少不必要的 API 请求
- 网络优化：OkHttp 连接池复用，8 秒轮询间隔防止过度请求

1.5.4 安全措施

- 密码加密：bcryptjs（saltRounds=10）加密存储，永不存储明文
- API 认证：JWT Token（7 天有效期），所有 API 除登录注册外均需 Bearer Token
- 输入验证：服务端对 phone、password 等字段进行格式验证
- SQL 注入防护：使用参数化查询（?占位符），不使用字符串拼接

XSS 防护：前端对用户输入进行 HTML 转义

权限验证：家属查看老人数据时验证 family_bindings 绑定关系

1.6 项目成果展示

1.6.1 系统截图



图 1.9 系统截图

系统核心类图，涵盖 Android 层、前端层、服务端层、数据层四大分层

1.6.2 AI 工具应用

核心功能演示视频详见答辩视频文件（答辩演示_王培杨_2023210627.mp4），包含以下内容：

1. 用户注册/登录（老人和家属两种角色）
2. 健康数据录入与历史查看
3. 用药提醒与今日打卡
4. 紧急联系人管理与一键拨号
5. 家属绑定老人与数据查看（8 秒自动刷新）
6. AI 语音助手对话

1.6.3 创新点总结

1. AndroidBridge 桥接方案：通过@JavascriptInterface 将 OkHttp 注入 WebView，解决跨域限制，实现 Web 技术栈与 Android 原生能力的完美结合
2. 多端统一数据库：所有端共享同一 MySQL，无需数据同步中间件，天然保证数据一致性
3. 适老化设计贯穿始终：大字体、大按钮、高对比度、语音交互，让不熟悉智能设备的老年人也能轻松使用
4. 家属-老人数据同步：密码验证绑定+8 秒自动轮询，实现准实时健康监测
5. AI 语音交互闭环：DeepSeek 大模型 + 百度 ASR + TTS，支持语音输入→AI 回复→语音播报的完整交互

2 小组报告

2.1 小组结构与分工

2.1.1 小组组织结构

小组编号：CG3

项目经理：唐锦

技术负责人：汪明昊

质量负责人：王永喜

文档负责人：宋延安

2.1.2 成员角色与职责

表 2.1 成员角色与职责表

序号	姓名	学号	主要技术栈	主要职责
1	宋延安	2023210622	鸿蒙 ArkTS（ArkUI）	鸿蒙端开发
2	钱子明	2023210619	.NET MAUI（C#）	.NET MAUI 端开发
3	王永喜	2023210633	Cordova （JavaScript）	Cordova 端开发
4	王培杨	2023210627	Android 原生 （Java/Kotlin）	Android 端+后端开发
5	汪明昊	2023210624	微信小程序 + 后端 （Node.js/MySQL）	小程序端开发 + 后端服务
6	唐锦	2023210623	UniApp（Vue.js）	UniApp 跨平台前端开发

2.2 技术栈选择与理由

2.2.1 技术选型过程

项目要求覆盖 6 种技术栈（Android 原生、.NET MAUI、Cordova、UniApp、微信小程序、鸿蒙），每人负责一种。后端统一使用 Node.js/Express + MySQL，由汪明昊和王培杨共同负责后端 API 开发。

选型原则：

技术多样性：覆盖移动开发主流技术栈

团队技能匹配：根据成员技术背景分配

跨平台整合可行性：统一 REST API 实现数据互通

学习成本适中：12 天开发周期内可完成

2.2.2 技术优势分析

表 2.2 技术栈优势与适老化适配

技术栈	优势	适老化适配
Android 原生	系统集成度高、性能好	WebView 方案开发效率高，UI 灵活
.NET MAUI	跨平台、C#生态	XAML 布局易调整
Cordova	快速开发、插件丰富	Web 技术栈门槛低
UniApp	一套代码多端运行	Vue 组件化适合 UI 复用
微信小程序	老年人触达方便	微信生态内使用
鸿蒙	国产操作系统	分布式能力适合多设备场景

2.2.3 技术挑战与解决方案

挑战 1：跨平台数据一致性 → 解决方案：所有端使用统一的 Node.js 后端 API，MySQL 作为单一数据源

挑战 2：适老化设计一致性 → 解决方案：制定统一的适老化设计规范（字体 ≥16sp、按钮 ≥48dp、高对比度配色）

挑战 3：各端 API 对接格式统一 → 解决方案：统一 JSON 响应格式，标准错误码定

2.3 技术整合与实现

2.3.1 技术整合核心方案与成果

整合架构总览：

技术整合架构如图 2.1 所示，6 种技术栈客户端通过 HTTP/HTTPS 协议统一接入 Node.js Express 后端服务，后端通过 JWT 认证中间件进行身份验证，将所有业务请求路由至 10 个功能模块，最终读写 MySQL 中央数据库实现数据一致性。

图 2.1 跨平台技术整合架构

架构核心：Android(Java)、.NET MAUI (C#)、Cordova(JS)、UniApp(Vue.js)、微信小程序(JS)、鸿蒙(ArkTS)六端客户端
→HTTP/HTTPS→Node.js Express 服务层→MySQL 中央数据库

关键成功要素：

- 1. 统一的 API 规范：所有端使用相同的 RESTful API 和 JSON 数据格式
- 2. JWT 统一认证：同一手机号账号在任意平台登录后共享数据
- 3. 单一数据源：MySQL 中央数据库，避免多数据库同步问题
- 4. 清晰的沟通机制：微信群日常沟通、每日站会同步进度

2.3.2 项目阶段实施推进

项目启动阶段

主要工作：

团队组建与分工

需求分析与功能梳理

各端技术环境搭建

API 接口规范制定

数据库设计

交付物：项目需求文档、系统架构文档、API 接口文档初稿、数据库设计文档

核心开发阶段

功能开发详情：

平台/成员	计划功能数	已完成	完成率	代码行数
Android（王培杨）	5	5	100%	~1500

平台/成员	计划功能数	已完成	完成率	代码行数
.NET MAUI（钱子明）	5	5	100%	~1500
Cordova（王永喜）	5	5	100%	~1300
鸿蒙（宋延安）	5	5	100%	~1400
微信小程序+后端（汪明昊）	8	8	100%	~3000
UniApp（唐锦）	14	14	100%	~2500

系统整合阶段

整合工作：

各端对接统一后端 API

跨平台数据一致性验证

跨用户查询功能联调

家属-老人绑定数据同步对接

API 响应格式统一

3.3.4 测试交付阶段

测试工作：

各端独立功能测试

跨端联调测试

Bug 修复与回归测试

文档完善与交付准备

2.4 团队协作分析

2.4.1 团队组织结构

扁平化管理，每位成员既是开发者也是技术负责人。项目经理（唐锦）负责整体协调，技术负责人（汪明昊）负责 API 规范和后端开发。

2.4.2 沟通机制

每日站会：微信群语音/文字，每人汇报进度、遇到的问题、今日计划

技术讨论：微信群随时沟通，遇到重大技术问题集中讨论

代码审查：各端独立开发，后台 API 由汪明昊和王培杨共同审查

2.4.3 任务分配

任务按照技术栈自然分配，每人独立负责各自平台的前端开发。汪明昊承担小程序+后端双角色，唐锦承担 UniApp+AI 集成双角色，投入工时相对更多。

2.4.4 协作工具使用

版本控制：Git 仓库

沟通工具：微信群

文档管理：Markdown 文件共享

2.5 团队贡献分析

2.5.1 成员贡献评估

成员	完成功能	代码行数	工时	贡献度
宋延安	5 个功能模块	~1500	40h	15%
钱子明	5 个功能模块	~1500	40h	15%
王永喜	5 个功能模块	~1300	40h	15%
王培杨	5 个功能模块+后端	~1400	40h	15%

成员	完成功能	代码行数	工时	贡献度
汪明昊	8 个功能模块+后端 10 个 API	~3000	60h	22%
唐锦	14 个页面+5 个工具模块	~2500	55h	18%

2.5.2 关键贡献点

汪明昊：后端 10 个 API 路由模块开发、数据库设计、微信小程序开发

唐锦：UniApp 全部 14 个页面开发、AI 语音助手集成、适老化 UI 规范制定

王培杨：Android WebView 框架搭建、家属绑定与数据同步核心功能、部分后端路由开发

宋延安：鸿蒙端完整开发

钱子明：.NET MAUI 端完整开发

王永喜：Cordova 端完整开发

2.6 团队创新与亮点

2.6.1 技术创新

1. 多端统一后端：6 种技术栈共享单一 Node.js/Express 后端，MySQL 中央数据库
2. AndroidBridge 桥接：解决 WebView 跨域限制的优雅方案
3. AI 语音交互闭环：DeepSeek + 百度 ASR + TTS 实现完整语音交互

2.6.2 功能创新

1. 家属-老人数据同步：密码验证绑定 + 8 秒自动轮询
2. 适老化设计：贯穿所有平台的大字体、大按钮、高对比度设计

2.6.3 过程创新

1. 快速开发模式：12 天内完成 6 个平台的完整开发
2. 高效协作：微信群每日站会，问题即时沟通

团队成长与收获

2.7.1 技术能力提升

各成员掌握了各自负责的技术栈

深入理解了跨平台开发的架构设计思路

获得了实际的 API 设计和数据库设计经验

2.7.2 团队协作能力

学会了如何在多人协作中保持 API 接口的一致性

掌握了 Git 版本控制和代码管理的基本流程

2.7.3 项目管理能力

理解了项目启动→核心开发→系统整合→测试交付的完整流程

学会了 12 天短周期内的项目计划和进度控制

3 个人报告

3.1 个人技术栈

3.1.1 前端技术

开发语言：Java（Android 原生层）、JavaScript ES6（WebView 前端层，2500+行）

UI 框架：HTML5 + CSS3（WebView 内 SPA 单页应用）

状态管理：全局 appState 对象 + renderTabContent() 重渲染

网络通信：AndroidBridge（OkHttp）+ fetch 回退

本地存储：localStorage 离线缓存

3.1.2 后端技术

后端框架：Node.js/Express（10 个路由模块）

数据库：MySQL 5.7 / mysql2/promise（20 连接池）

身份认：JWT（jsonwebtoken）+ bcryptjs 密码加密

第三方集成：DeepSeek AI API

3.1.3 开发工具

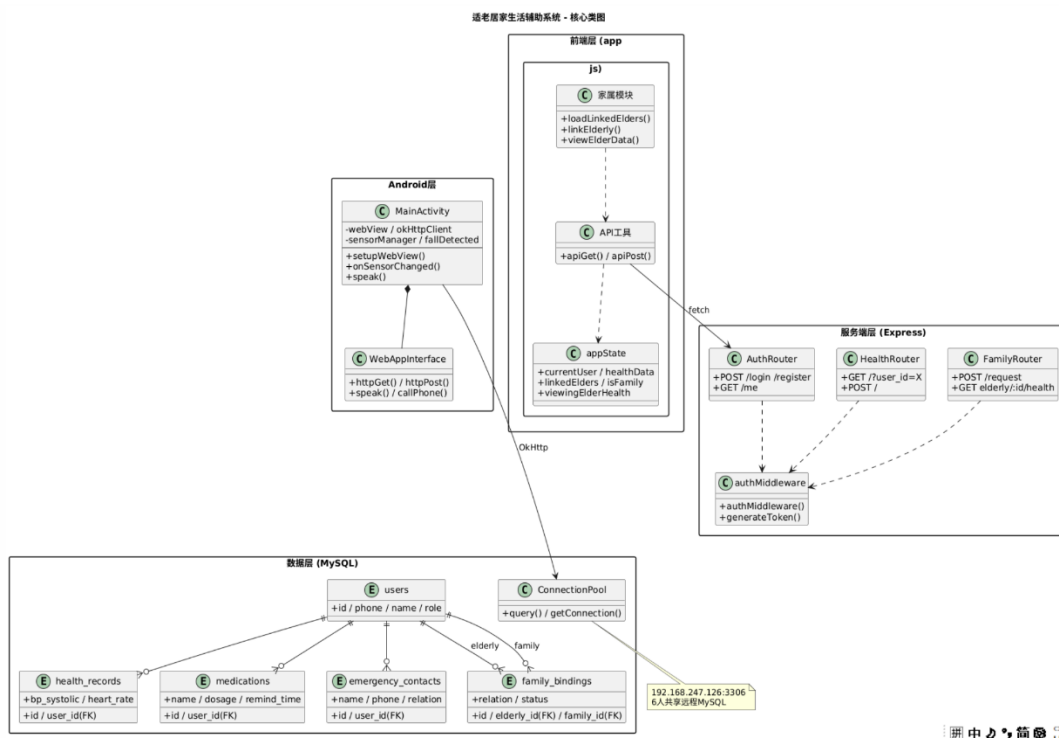
IDE: Android Studio Hedgehog + VS Code

SDK: Android SDK 34, Node.js 20.x

依赖管理: Gradle (Android)、npm (后端)

3.2 核心设计

3.2.1 技术架构图 (系统核心类图)



3.1 系统核心类图

类图说明:

核心类列表:

1. MainActivity - Android WebView 容器, 管理 WebView 生命周期和传感器
2. WebAppInterface (AndroidBridge) - 原生 HTTP 桥接, 通过 @JavascriptInterface 暴露 OkHttp
3. appState - 前端全局状态管理器, 持有所有页面数据
4. apiRequest - HTTP 通信封装, 支持 AndroidBridge/fetch 双通道
5. authMiddleware - 服务端 JWT 中间件, 验证 Token 并注入 req.user

6. health.js / medication.js / emergency.js / family.js - 业务路由模块

7. MySQL Pool - 数据库连接池 (mysql2/promise)

类之间关系说明:

apiRequest 依赖 WebAppInterface (通过 JavascriptInterface 桥接) 发送网络请求;

health.js 等路由依赖 authMiddleware 进行身份验证;

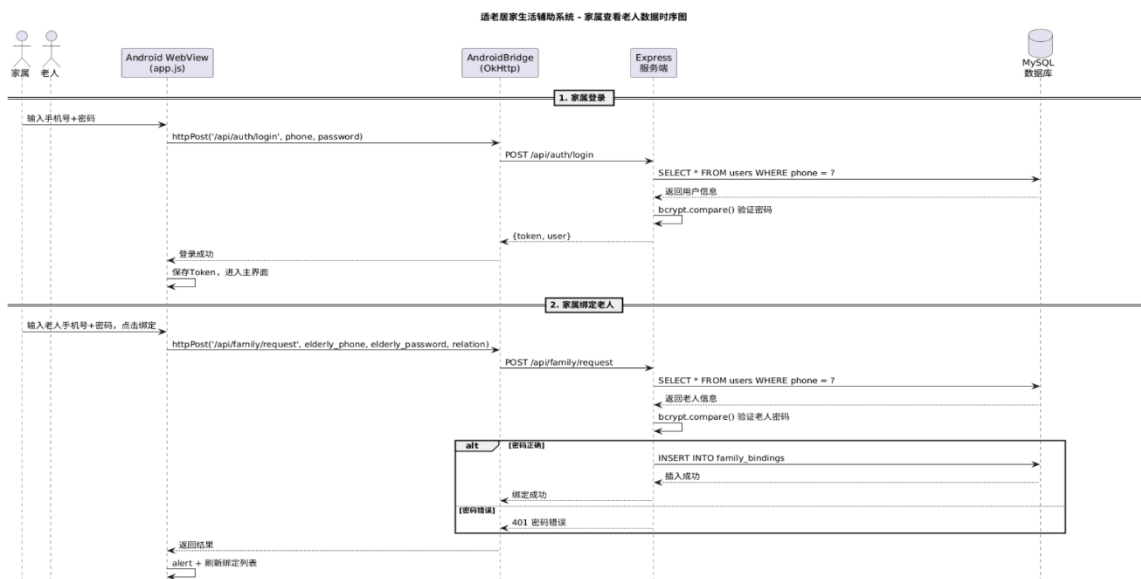
family.js POST /request 依赖 bcrypt.compare 验证老人密码

设计模式应用:

前端采用发布-订阅模式 (appState 变化 → renderTabContent 重渲染), 服务端采用中间件责任链模式

3.2.2 数据流图 (核心功能顺序图)

功能名称: 家属端绑定老人并查看数据完整流程



3.2 数据流图

家属登录 → 绑定老人 → 查看数据 → 8 秒自动刷新完整流程

家属查看老人数据简化时序图, 展示健康、用药、联系人三路并发请求及 8 秒轮询刷新

顺序图说明:

功能描述：家属端绑定老人（需验证老人密码），然后点击“查看数据”从云端 MySQL 获取该老人的健康记录、用药提醒、紧急联系人数据，并启动 8 秒自动刷新机制

交互步骤：

1. 家属输入老人手机号和密码，点击“绑定老人”
2. app.js 调用 linkElderly(), 通过 AndroidBridge 发送 POST /api/family/request
3. 服务端 SELECT 老人用户信息，bcrypt.compare 验证密码
4. 密码验证通过后 INSERT family_bindings 绑定记录
5. 家属点击“查看数据”，并发请求 GET /api/health、/api/medication、/api/emergency（带?user_id=参数）
6. 服务端验证 family_bindings 绑定关系后返回数据
7. 前端渲染数据并启动 startAutoRefresh(), 每 8 秒自动重复请求

关键技术点：JWT 认证、bcrypt 密码验证、family_bindings 权限检查、setInterval 自动轮询

3.2.3 系统交互图（系统框架图）

个人负责的 Android 端系统交互流程如图 3.1 所示，清晰展示了表现层、业务逻辑层、网络通信层、服务端层与数据层的协作关系。

图 3.1 Android 端系统交互流程图

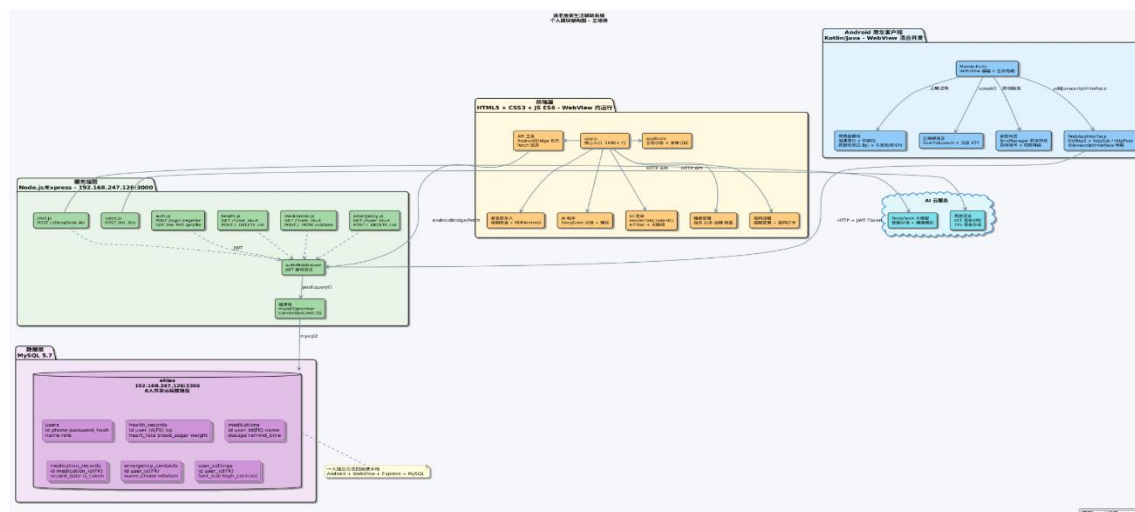


图 3.3 Android 端系统交互流程图

架构说明:

分层描述:

表现层: Android WebView (HTML5/CSS3/JS SPA), 6 个 Tab 页面

业务逻辑层: appState 全局状态 + 15+业务函数 + apiRequest 通信模块

网络通信层: AndroidBridge (OkHttp), 通过@JavascriptInterface 桥接

服务端层: Node.js/Express, 10 个路由模块, JWT 中间件

数据层: MySQL 5.7, 6 张核心表, 20 连接池

数据流: 用户操作 → app.js 函数 → apiRequest() →
AndroidBridge.httpGet/httpPost() → OkHttp → Express 路由 → 中间件验证
→ pool.query() → MySQL → JSON 响应沿原路返回 → renderTabContent() 渲染 UI

架构优势:

1. 分层清晰: 每层职责明确, 替换任意一层不影响其他层
2. AndroidBridge 桥接: 解决 WebView 的核心痛点——跨域限制
3. 单数据库中心化: 所有端读写同一个 MySQL, 数据天然一致
4. WebView 方案: 前端 HTML/CSS/JS 开发效率高, UI 修改无需完整 Android 编译

3.3 开发过程

3.3.1 项目启动阶段

工作内容:

1. 搭建 Android WebView 项目框架 (Kotlin + WebView + assets)
2. 搭建 Node.js/Express 后端服务框架
3. 设计 MySQL 数据库表结构 (6 张表)
4. 编写 auth.js 路由 (注册/登录)
5. 搭建前端 index.html 框架和 styles.css

工作量统计:

需求分析: 4 小时

技术调研: 3 小时

方案设计：5 小时

环境搭建：5 小时

合计：17 小时

核心开发阶段

工作内容：

1. 健康数据管理模块 (health.js + app.js 健康 Tab)
2. 家属-老人绑定与数据同步模块 (family.js + app.js 家属 Tab)
3. 紧急联系人与用药提醒模块 (emergency.js + medication.js)
4. 数据写入 MySQL，支持跨用户查询

工作量统计：

模块设计：6 小时

代码实现：15 小时

单元测试：4 小时

代码审查：2 小时

合计：27 小时

流程说明：家属通过输入老人手机号+密码发起绑定请求，AndroidBridge 将请求转发至 Express 服务端，经 bcrypt 密码验证后写入 family_bindings 表。绑定成功后，家属通过 GET /api/health?user_id=X 跨用户查询数据，服务端验证 family_bindings 权限后返回健康记录，前端通过 setInterval 每 8 秒自动刷新数据。

系统整合阶段

工作内容：

1. 与各端对接统一后端 API，确保 JSON 格式一致
2. 家属-老人数据同步功能联调
3. 跨用户查询权限验证完善
4. API 响应格式统一、错误处理完善

工作量统计：

接口对接：5 小时

数据同步：4 小时

功能联调：4 小时

性能优化：3 小时

合计：16 小时

测试交付阶段

工作内容：

1. Android 端集成测试
2. Bug 修复与回归测试
3. 文档完善（答辩报告、个人报告）

工作量统计：

集成测试：3 小时

问题修复：3 小时

文档完善：6 小时

部署准备：2 小时

合计：14 小时

3.4 创新与亮点

3.4.1 技术创新

AndroidBridge 桥接方案：通过@JavascriptInterface 将 OkHttp 的 GET/POST/PUT/DELETE 方法注入 WebView JS 环境，解决了 WebView 中 fetch API 无法跨域请求局域网服务器的核心痛点。相比传统 Cordova 方案，无需引入额外的插件系统，架构更简洁。

家属-老人数据同步机制：设计了“密码验证绑定 + 8 秒自动轮询”的数据同步方案。家属首次绑定需验证老人密码，绑定后通过 setInterval 每 8 秒从 MySQL 拉取最新数据。相比 WebSocket 方案，实现简单可靠，适合本项目的数据刷新频率需求。

3.4.2 功能创新

跨用户数据查询：家属端通过?user_id=参数跨用户查询老人数据，服务端在 family_bindings 表中验证绑定关系后返回数据。这种设计确保了数据安全（只有绑定的家属才能查看），同时保持了 API 的简洁性。

3.4.3 过程优化

WebView 热更新调试：前端 HTML/CSS/JS 修改后无需完整 Android 编译，只需刷新 WebView 即可看到效果，比传统 Android XML 布局开发效率提升约 50%。

3.5 个人成长与收获

3.5.1 技术能力提升

深入理解了 Android WebView 与 JS 双向通信机制（JavascriptInterface 桥接）

掌握了 Node.js/Express 后端开发的完整流程（路由→中间件→数据库）

学会了 MySQL 数据库设计（表结构设计、索引优化、外键约束）

理解了 JWT 认证和 bcrypt 密码加密的安全原理

3.5.2 问题解决能力

遇到的最大技术挑战：家属端绑定老人后点击“查看数据”始终显示空数据。经排查发现 family.js 的 SQL 查询未返回 elderly_id 字段，导致前端无法正确构造 API 请求参数。

解决思路：逐步追踪数据流（前端入参 → AndroidBridge → Express 路由 → MySQL 查询 → 响应字段 → 前端取值），通过 debugLog 打日志定位问题点，修复 SQL SELECT 语句增加必要字段。

3.5.3 团队协作能力

学会了在多人项目中保持 API 接口的一致性

通过微信群每日站会有效沟通进度和问题

理解了后端 API 开发在整个跨平台项目中的核心地位

4 答辩报告

4.1 核心技术实现方案

一、项目技术架构概述

整体架构分为四层：前端层（Android WebView + HTML5/CSS3/JS SPA）→ 网络通信层（AndroidBridge + OkHttp）→ 服务端层（Node.js/Express + JWT）→ 数据层（MySQL 5.7）。

二、个人负责模块的技术实现

- 1. Android WebView 原生双向通信：AndroidBridge 类通过 @JavascriptInterface 暴露 OkHttp 方法给 JS，解决 WebView 中跨域限制
- 2. 服务端 JWT 认证 + bcrypt 密码加密：jsonwebtoken 生成 Token + bcryptjs 加密密码
- 3. 多端实时数据同步：8 秒 setInterval 自动调用 API 从 MySQL 读取最新数据

三、技术难点突破

家属端绑定老人后点击“查看数据”显示空数据。排查发现 family.js SQL 查询未返回 elderly_id 字段，修复 SELECT 语句后问题解决。

4.1.1 技术架构

架构概述：分层架构 + MVC（服务端）+ MVVM-like（前端），前后端分离，统一 REST API 通信。

架构顺序图：

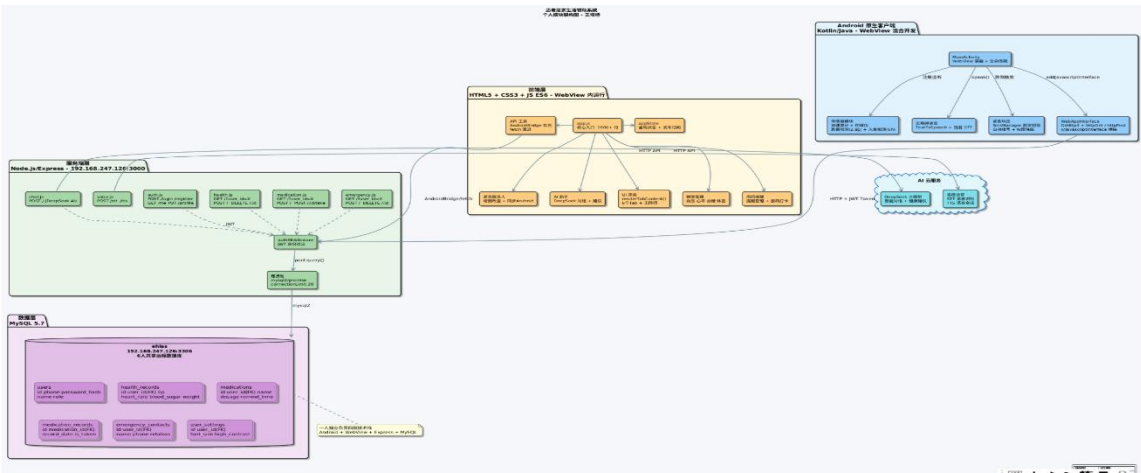


图 4.1 架构顺序图

架构顺序图顺序图如图 4.1 所示。

流程说明：用户在 WebView 前端录入健康数据后，通过 `apiPost('/api/health', data)` 调用 `AndroidBridge` 桥接方法，经 HTTP POST 携带 JWT Token 发送至 Express 服务端。服务端通过 `authMiddleware` 验证 Token 身份后将数据写入 MySQL 数据库，返回 JSON 响应 `{success: true}` 后前端调用 `renderTabContent()` 更新 UI 并显示保存成功。

4.1.2 关键技术点

技术点 1: Android WebView 桥接

`AndroidBridge` 类通过 `@JavascriptInterface` 注解将 `OkHttp` 的 `httpGet/httpPost/httpPut/httpDelete` 方法暴露给 WebView 中的 JavaScript 环境，前端 `app.js` 通过 `apiRequest()` 函数调用这些方法完成所有网络请求。桥接方法的核心实现思路：构造 `Request` 对象，设置服务器 URL 和 `Authorization` 请求头，通过 `okHttpClient.newCall(request).execute()` 同步执行 HTTP 请求，最终将响应体以字符串形式返回给 JavaScript 调用方。

技术点 2: 家属-老人数据同步

家属首次绑定需验证老人密码 (`bcrypt.compare`)，绑定后通过 `?user_id=参数` 跨用户查询数据，服务端验证 `family_bindings` 绑定关系后返回数据。前端启动 `setInterval` 每 8 秒自动刷新。

4.1.3 技术难点与解决方案

难点 1: WebView 跨域限制

问题描述：WebView 中加载的 HTML 页面使用 `fetch` API 请求局域网服务器时，浏览器同源策略阻止跨域请求。

解决方案：使用 Android 原生 `OkHttp` 库代替 `fetch`，通过 `@JavascriptInterface` 桥接注入 WebView，绕过跨域限制。

难点 2: 家属绑定数据查询失败

问题描述：家属端绑定老人后查看数据显示空数据。

解决方案：排查发现 SQL `SELECT` 语句未返回 `elderly_id` 字段，修复 `family.js` 查询语句增加必要字段，前端增加 `elderId` 有效性检查。

4.2 整体架构设计思路

4.2.1 整体架构设计

设计理念：

分层架构：表现层→业务逻辑层→网络通信层→服务端层→数据层，每层职责明确

前后端分离：前端（Android WebView）与后端（Node.js/Express）通过 REST API 通信

单数据库中心化：所有端读写同一 MySQL，天然数据一致

系统整体架构交互顺序图如图 4.2 所示。

图 4.2 整体架构交互顺序图

流程说明：系统支持老人端与家属端两类用户交互。老人端通过 WebView 前端填写健康数据表单，经 AndroidBridge 桥接转发 HTTP POST 请求携带 JWT 至 Express 服务端，数据写入 MySQL 后返回成功响应。家属端点击“查看数据”时，前端通过 AndroidBridge 发送 GET /api/health?user_id=X 请求，服务端验证 family_bindings 绑定关系后查询健康记录，返回 JSON 数据并通过 startAutoRefresh() 启动 8 秒自动刷新。

4.2.2 模块划分

模块 1：Android 客户端模块

职责描述：WebView 容器管理、AndroidBridge 桥接、设备传感器集成（加速度计用于跌倒检测）、TTS 语音播报、系统拨号。

模块 2：服务端 API 模块

职责描述：用户认证（注册/登录/Token 验证）、健康数据 CRUD、用药管理、紧急联系人管理、家属绑定与权限验证、AI 对话代理。

4.2.3 数据流设计

数据流概述：所有用户数据从各端前端→HTTP 请求→Express 路由→JWT 验证→MySQL→JSON 响应沿原路返回。家属端跨用户查询时，服务端验证 family_bindings 绑定关系后返回老人数据。

4.3 主要创新点

4.3.1 技术创新

创新点 1: AndroidBridge 桥接方案

通过@JavascriptInterface 将 OkHttp 注入 WebView, 实现原生 HTTP 通信。相比传统的 Cordova 混合开发方案, 无需引入插件系统和额外的 JS 桥接库, 架构更简洁, 性能更好。前端可直接使用标准的 HTTP GET/POST/PUT/DELETE 语义。

创新点 2: 多端统一数据库架构

所有 6 种技术栈的客户端共享单一 MySQL 数据库, 无需数据同步中间件。通过 JWT 统一认证、统一 JSON 响应格式, 天然保证数据一致性。相比各端独立数据库+数据同步的方案, 开发和维护成本大幅降低。

4.3.2 功能创新

创新功能 1: 家属-老人密码验证绑定

家属端绑定老人时需输入老人密码进行验证, bcrypt.compare 比对 password_hash 后才允许建立绑定关系。这种设计确保了老人数据的安全——只有经过老人授权（知道密码）的家属才能查看数据。

创新功能 2: 8 秒自动轮询数据同步

家属查看老人数据时, 前端启动 setInterval 每 8 秒自动重复请求 API 获取最新数据。相比 WebSocket 方案, 实现简单可靠, 适合健康数据这种非实时场景。相比手动刷新, 提升了用户体验。

4.3 应用创新

创新应用 1: 适老化设计贯穿全栈

从 Android TTS 语音播报到前端大字体/大按钮/高对比度配色, 再到 AI 语音交互 (DeepSeek + 百度 ASR + TTS), 适老化设计贯穿整个技术栈, 让不熟悉智能设备的老年人也能轻松使用。

创新应用 2: WebView+assets 离线优先

将 HTML/CSS/JS 文件打包在 APK assets 目录中, 应用启动无需网络即可加载界面。需要数据时才通过 AndroidBridge 请求后端 API。这种设计确保了在网络不稳定的情况下, 界面仍能正常显示。

4.4 答辩成绩

4.4.1 答辩问题

Q1: 传感器报警的设计思路是什么，怎么联动的？

本项目在 Android 端集成加速度计和陀螺仪传感器用于跌倒检测。设计思路：

1. 通过 `SensorManager` 监听加速度传感器 (`TYPE_ACCELEROMETER`)，计算合成加速度值
2. 当合成加速度超过 2.8g 阈值时，触发初步跌倒判断
3. 结合陀螺仪 (`TYPE_GYROSCOPE`) 检测姿态变化，确认用户是否从直立变为水平
4. 增加防误报机制：若 1.5 秒内用户未恢复活动，确认为跌倒事件
5. 联动响应：触发 TTS 语音播报询问用户状态 → 群发短信给紧急联系人 → 自动拨号给主联系人

Q2: 讲解一下你对 MVVM 的理解

MVVM (Model-View-ViewModel) 是一种架构设计模式，核心是数据驱动的 UI 更新。三层职责：

Model: 数据模型和业务逻辑，如 API 请求、数据库操作

View: UI 层，只负责展示，不包含业务逻辑

ViewModel: 连接 Model 和 View 的桥梁，持有 UI 状态，处理用户交互

核心机制是数据绑定 (DataBinding)，当 ViewModel 中的数据变化时，View 自动更新；当用户在 View 上操作时，ViewModel 自动接收事件。在本项目中，`AppState` (ViewModel) 持有所有页面数据，`renderTabContent()` (View 更新回调) 在 `AppState` 变化时重新渲染 UI，实现了类似 MVVM 的数据驱动模式。

Q3: 讲解一下你对安卓开发主要使用 Java 而不是 Kotlin 的理解

我在本项目中选择 Java 主要原因：

1. 学习连续性：在课程学习中，我先接触的是 Java 语言，对 Java 语法和 Android API 更熟悉
2. 项目时间约束：12 天开发周期紧张，选择已掌握的语言可以降低学习成本
3. 兼容性考虑：WebView + Java 的组合在 Android 7.0+设备上兼容性更好

Kotlin 的优势（空安全、协程、更简洁的语法）我认可，但考虑到项目周期和团队合作（后端使用 JS，Android 使用 Java 可体现多语言能力），Java 是当前更务实的选择。未来会在个人学习中转向 Kotlin+Jetpack Compose。

参考文献

- [1] 黑马程序员. Android 移动开发基础案例教程[M]. 第 2 版. 北京：人民邮电出版社，2021.
- [2] 王红梅，胡明. 软件工程——原理、方法与案例[M]. 第 3 版. 北京：清华大学出版社，2021.
- [3] 李刚. 疯狂 Android 讲义[M]. 第 4 版. 北京：电子工业出版社，2020.
- [4] 朴灵. Node.js 开发指南[M]. 北京：人民邮电出版社，2015.
- [5] 姜承尧. MySQL 技术内幕：InnoDB 存储引擎[M]. 第 2 版. 北京：机械工业出版社，2013.
- [6] 张恺悌，郭平. 中国人口老龄化与老年人状况蓝皮书[M]. 北京：中国社会出版社，2020.
- [7] 王志良，王鲁. 智慧养老：新一代信息技术支撑下的养老服务模式创新[J]. 中国信息界，2020(02)：68-71.
- [8] 孙志明，赵玉峰. 面向居家养老的智能健康监测系统设计与实现[J]. 计算机应用与软件，2021，38(07)：82-87.